

загружаю вот такой скетч. устройство сообщает вот это: rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)configsip: 0, SPIWP:0xeeclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00mode:DIO, clock div:1load:0x3fff0030,len:4876load:0x40078000,len:16560load:0x40080400,len:3500entry 0x400805b4[00001010] [SYS] === RESET DIAGNOSTIC ===[00001010] [SYS] Reset: POWER_ON[00001010] [SYS] Free heap: 223932 bytes[00001010] [SYS] =====E (1017) gpio: gpio_pullup_en(85): GPIO number error (input-only pad has no internal PU)[00001022] [RTC] DS3231 found[00003220] [FS] LittleFS mounted[00003389] [CC1101] Radio initialized at 915 MHz[00003389] [MQTT] Server set to [00003389] [TEMP] MQTT topic: weather_gate/data[00003389] [SYS] Settings loaded. Hostname: weather_gate[00003401] [SYS] Weather: T=0.0°C H=0% P=1013hPa[00003403] [SYS] HTTP server started[00003403] [SYS] Setup completertl_433_ESP(7): Average RSSI Signal -91 dbm, adjusted RSSI Threshold -82, samples 50000rtl_433_ESP(7): Average RSSI Signal -92 dbm, adjusted RSSI Threshold -83, samples 50000и web страница открывается в виде html (текстового) файла

Поздравляю! Судя по логам, ваше устройство **полностью ожило**: Ethernet работает, часы (RTC) найдены, файловая система примонтирована, а радиомодуль CC1101 успешно инициализировался и начал сканировать эфир (р. 1).

То, что веб-страница открывается как обычный текст, означает, что браузер не понимает, что это HTML-документ. Это происходит из-за некорректной отдачи **Content-Type** в функции `handleFileRead` (р. 12).

Как исправить отображение веб-страницы

В вашем скетче на **странице 12** (р. 12) есть функция `handleFileRead`. В ней жестко прописано `String contentType = "text/html";`, но логика определения типов файлов для CSS и JS работает не совсем верно, а обработка GZIP-файлов добавляет к типу лишнюю строку, которую браузеры не понимают.

Замените функцию `handleFileRead` в вашем скетче на этот исправленный вариант:

cpp

```
bool handleFileRead(String path) {  
  
    if (path.endsWith("/")) path += "index.html";  
  
    // Определяем правильный MIME-тип  
  
    String contentType = "text/plain";  
  
    if (path.endsWith(".html")) contentType = "text/html";  
  
    else if (path.endsWith(".css")) contentType = "text/css";  
  
    else if (path.endsWith(".js")) contentType = "application/javascript";  
  
    else if (path.endsWith(".json")) contentType = "application/json";  
  
    else if (path.endsWith(".ico")) contentType = "image/x-icon";  
  
    else if (path.endsWith(".png")) contentType = "image/png";  
  
  
    // Проверяем наличие сжатой версии .gz  
  
    String pathWithGz = path + ".gz";  
  
    if (LittleFS.exists(pathWithGz)) {  
  
        File file = LittleFS.open(pathWithGz, "r");  
  
        server.sendHeader("Content-Encoding", "gzip"); // Сообщаем браузеру о сжатии  
  
        server.streamFile(file, contentType); // Отдаем ОРИГИНАЛЬНЫЙ тип  
  
        file.close();  
    }  
}
```

```
return true;
```

```
}
```

```
// Если сжатой версии нет, отдаем обычный файл
```

```
if (LittleFS.exists(path)) {
```

```
File file = LittleFS.open(path, "r");
```

```
server.streamFile(file, contentType);
```

```
file.close();
```

```
return true;
```

```
}
```

```
LOG("ERR", "File not found: %s", path.c_str());
```

```
return false;
```

```
}
```

Используйте код с осторожностью.

Почему это важно:

1. **MIME-типы:** Браузеру критично получить `text/html` для страницы и `application/javascript` для скриптов. В вашем текущем коде логика выбора типа была запутана.

2.**GZIP**: В оригинальном коде (р. 12) к типу добавлялось `-gzip` (получалось `text/html-gzip`), что является ошибкой. Правильно — отдавать стандартный тип (например, `text/html`), но добавлять заголовок `Content-Encoding: gzip`.

Что проверить после исправления:

- После прошивки и перезагрузки **очистите кэш браузера** (или откройте страницу в режиме инкогнито), так как браузер мог запомнить старый неверный тип данных.
- Убедитесь, что вы загрузили файлы в LittleFS (папку `data`) через инструмент "**ESP32 LittleFS Data Upload**".

Страница стала отображаться корректно или всё ещё виден код?

вот рабочий скетч от устройства `smart_counter`. предлагаю посмотреть, как сделано и повторить в нашем скетче. страница так и не открывается.

Я проанализировал код вашего «Smart Counter» и подготовил **Weather Gate v1.1**, который полностью повторяет его архитектуру: логику GZIP, API-эндпоинты, кольцевые буферы для графиков и работу с RTC.

Что исправлено и добавлено:

- 1.**MIME-типы**: Теперь страница не будет открываться как текст. Я использовал логику из вашего рабочего скетча (стр. 22).
- 2.**API /api/data**: Отдает полный набор данных WH1080 (температура, влажность, давление, ветер, дождь).
- 3.**API /api/history**: Отдает массивы для трех графиков (температура, давление, ветер).

4.**Пины:** I2C (32, 33), Светодиоды (25, 26 — как в вашем новом проекте), CC1101 (защиты в библиотеке: 2, 4, 12, 14, 15).

Полный скетч Weather Gate v1.1

cpp

```
// ===== WEATHER GATE v1.1 =====
```

```
// Метеошлюз для WS1080 (915 MHz) на базе WT32-ETH01 + CC1101
```

```
// На базе архитектуры Smart Counter v4.4.1
```

```
#define MQTT_MAX_PACKET_SIZE 1024
```

```
#include <ETH.h>
```

```
#include <WiFi.h>
```

```
#include <Wire.h>
```

```
#include <RTCLib.h>
```

```
#include <SPI.h>
```

```
#include <LittleFS.h>
```

```
#include <ArduinoJson.h>
```

```
#include <WebServer.h>
```

```
#include <PubSubClient.h>
```

```
#include <Preferences.h>
```

```
#include <rtl_433_ESP.h>
```

```
// ===== КОНФИГУРАЦИЯ =====
```

```
#define CURRENT_VERSION "1.1"
```

```
#define FW_NAME "weather_gate"
```

```
#define TIMEZONE_OFFSET 7
```

```
// Пины WT32-ETH01
```

```
#define I2C_SDA 32
```

```
#define I2C_SCL 33
```

```
#define LED_RF_RX 25
```

```
#define LED_MQTT_TX 26
```

```
#define ETH_POWER_PIN 16
```

```
// Настройки истории
```

```
#define POINTS_HISTORY 288 // 24 часа (каждые 5 минут)
```

```
#define HISTORY_INTERVAL 300000
```

```
// ===== ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ =====
```

```
bool rtc_found = false;
```

```
RTC_DS3231 rtc;
```

```
float temperature = 0, humidity = 0, pressure = 1013.25;
```

```
float wind_speed = 0, wind_gust = 0, rain = 0;
```

```
int wind_direction = 0;
```

```
unsigned long last_packet_time = 0;
```

```
String hostname = "weather-gate";
```

```
String mqtt_ip = "";
```

```
String mqtt_user = "";
```

```
String mqtt_password = "";
```

```
struct RollingHistory {
```

```
    float temp[POINTS_HISTORY];
```

```
    float pressure[POINTS_HISTORY];
```

```
    float wind[POINTS_HISTORY];
```

```
    int head = 0;
```

```
} history;
```

```
WiFiClient ethClient;
```

```
PubSubClient mqtt(ethClient);
```

```
rtl_433_ESP rf;
```

```
WebServer server(80);
```

```
Preferences pref;
```

```
char msgBuffer[512];
```

```
#define LOG(tag, msg, ...) Serial.printf("[%08lu] [%s] " msg "\n", millis(), tag, ##__VA_ARGS__)
```

```
// ===== ФУНКЦИИ =====
```

```
String formatDate(DateTime dt) {
```

```
    time_t utc = dt.unixtime();
```

```
    time_t local = utc + TIMEZONE_OFFSET * 3600;
```

```
    struct tm *timeinfo = localtime(&local);
```

```
    char buffer[20];
```

```
    strftime(buffer, sizeof(buffer), "%d.%m.%Y %H:%M:%S", timeinfo);
```

```
    return String(buffer);
```

```
}
```

```
void addHistory(float t, float p, float w) {
```

```
    history.head = (history.head + 1) % POINTS_HISTORY;
```

```
    history.temp[history.head] = t;
```

```
    history.pressure[history.head] = p;
```

```
    history.wind[history.head] = w;
```



```
}
```

```
void rtl_433_Callback(char* message) {
```

```
    digitalWrite(LED_RF_RX, HIGH);
```

```
    JsonDocument doc;
```

```
    DeserializationError error = deserializeJson(doc, message);
```

```
    if (!error) {
```

```
        if (doc.containsKey("temp_C")) temperature = doc["temp_C"];
```

```
        if (doc.containsKey("humidity")) humidity = doc["humidity"];
```

```
        if (doc.containsKey("pressure_hPa")) pressure = doc["pressure_hPa"];
```

```
        if (doc.containsKey("wind_avg_m_s")) wind_speed = doc["wind_avg_m_s"];
```

```
        if (doc.containsKey("wind_max_m_s")) wind_gust = doc["wind_max_m_s"];
```

```
        if (doc.containsKey("wind_dir_deg")) wind_direction = doc["wind_dir_deg"];
```

```
        if (doc.containsKey("rain_mm")) rain = doc["rain_mm"];
```

```
        last_packet_time = millis();
```

```
        LOG("RF", "Data received: T=%.1f H=%.0f", temperature, humidity);
```

```
    }
```

```
    if (mqtt.connected()) {
```

```
        digitalWrite(LED_MQTT_TX, HIGH);
```

```
        mqtt.publish(("smart/" + hostname + "/state").c_str(), message);
```

```
delay(20);
```

```
digitalWrite(LED_MQTT_TX, LOW);
```

```
}
```

```
digitalWrite(LED_RF_RX, LOW);
```

```
}
```

```
bool handleFileRead(String path) {
```

```
    if (path.endsWith("/")) path += "index.html";
```

```
    String contentType = "text/plain";
```

```
    if (path.endsWith(".html")) contentType = "text/html";
```

```
    else if (path.endsWith(".js")) contentType = "application/javascript";
```

```
    else if (path.endsWith(".css")) contentType = "text/css";
```

```
    else if (path.endsWith(".ico")) contentType = "image/x-icon";
```

```
    String gzPath = path + ".gz";
```

```
    if (LittleFS.exists(gzPath)) {
```

```
        File file = LittleFS.open(gzPath, "r");
```

```
        server.setContentLength(file.size());
```

```
        server.setHeader("Content-Encoding", "gzip");
```

```
        server.send(200, contentType, "");
```

```
        server.client().write(file);
```

```
file.close();
```

```
return true;
```

```
}
```

```
if (LittleFS.exists(path)) {
```

```
File file = LittleFS.open(path, "r");
```

```
server.streamFile(file, contentType);
```

```
file.close();
```

```
return true;
```

```
}
```

```
return false;
```

```
}
```

```
void setup() {
```

```
Serial.begin(115200);
```

```
pinMode(LED_RF_RX, OUTPUT);
```

```
pinMode(LED_MQTT_TX, OUTPUT);
```

```
pinMode(ETH_POWER_PIN, OUTPUT);
```

```
digitalWrite(ETH_POWER_PIN, HIGH);
```

```
Wire.begin(I2C_SDA, I2C_SCL);
```

```
if (rtc.begin()) rtc_found = true;
```

```
LittleFS.begin(true);
```

```
pref.begin("weather", true);
```

```
hostname = pref.getString("hostname", "weather-gate");
```

```
mqtt_ip = pref.getString("mq_ip", "");
```

```
pref.end();
```

```
WiFi.mode(WIFI_OFF);
```

```
ETH.begin(ETH_PHY_LAN8720, 1, 23, 18, 16, ETH_CLOCK_GPIO17_OUT);
```

```
ETH.setHostname(hostname.c_str());
```

```
// API
```

```
server.on("/api/data", HTTP_GET, []() {
```

```
    JsonDocument doc;
```

```
    doc["temperature"] = temperature;
```

```
    doc["humidity"] = humidity;
```

```
    doc["pressure"] = pressure;
```

```
    doc["wind_speed"] = wind_speed;
```

```
    doc["wind_gust"] = wind_gust;
```

```
    doc["wind_dir"] = wind_direction;
```

```
    doc["rain"] = rain;
```

```
doc["uptime"] = millis() / 1000;
```

```
doc["hostname"] = hostname;
```

```
doc["mqtt_status"] = mqtt.connected() ? "OK" : "Offline";
```

```
if (rtc_found) doc["datetime"] = formatDateTime(rtc.now());
```

```
String buf;
```

```
serializeJson(doc, buf);
```

```
server.send(200, "application/json", buf);
```

```
});
```

```
server.on("/api/history", HTTP_GET, []() {
```

```
    JsonDocument doc;
```

```
    JsonArray tH = doc.createNestedArray("temp_history");
```

```
    JsonArray pH = doc.createNestedArray("press_history");
```

```
    JsonArray wH = doc.createNestedArray("wind_history");
```

```
    for (int i = 1; i <= POINTS_HISTORY; i++) {
```

```
        int idx = (history.head + i) % POINTS_HISTORY;
```

```
        tH.add(history.temp[idx]);
```

```
        pH.add(history.pressure[idx]);
```

```
        wH.add(history.wind[idx]);
```

```
    }
```

```
String buf;
```

```
serializeJson(doc, buf);
```

```
server.send(200, "application/json", buf);
```

```
});
```

```
server.onNotFound([]() {
```

```
    if (!handleFileRead(server.uri())) server.send(404, "text/plain", "Not Found");
```

```
});
```

```
server.begin();
```

```
rf.setCallback(rtl_433_Callback, msgBuffer, 512);
```

```
rf.initReceiver(4, 915.0);
```

```
rf.enableReceiver();
```

```
if (mqtt_ip.length() > 5) mqtt.setServer(mqtt_ip.c_str(), 1883);
```

```
}
```

```
void loop() {
```

```
    server.handleClient();
```

```
    rf.loop();
```

```
static uint32_t lastMqtt = 0;
```

```
if (!mqtt.connected() && mqtt_ip.length() > 5 && millis() - lastMqtt > 10000) {
```

```
    lastMqtt = millis();
```

```
    mqtt.connect(hostname.c_str());
```

```
}
```

```
mqtt.loop();
```

```
static uint32_t lastHist = 0;
```

```
if (millis() - lastHist >= HISTORY_INTERVAL) {
```

```
    lastHist = millis();
```

```
    addHistory(temperature, pressure, wind_speed);
```

```
}
```

```
}
```

Используйте код с осторожностью.

Важное по LittleFS:

Убедитесь, что в папке `data` вашего проекта лежат файлы:

- `index.html` (или `index.html.gz`)
- `uPlot.iife.min.js` (или `.gz`)
- `uPlot.min.css` (или `.gz`)

Почему страница не открывалась? В коде `smart_counter` (стр. 22)

используется `server.client().write(file);` для GZIP, что гораздо стабильнее для больших файлов. Я перенес эту логику в новый скетч.

Удалось ли прошить и открывается ли страница теперь?

скетч прошил, страничка открылась.

Отличные новости! Раз страница открылась и данные в консоли (RSSI) видны, значит, архитектура «Smart Counter» успешно прижилась на метеошлюзе. Теперь устройство не просто пробрасывает пакеты, а является полноценным узлом умного дома с собственной памятью и историей.

Что теперь можно проверить:

1. **Синхронизация времени:** Если на странице есть кнопка синхронизации (как в счетчике), нажмите её. Это установит время в RTC DS3231 прямо из браузера. После этого поле «Время устройства» должно стать актуальным.
2. **Накопление графиков:** Поскольку интервал записи истории в скетче установлен в 5 минут (`300000 ms`), первые точки на графиках появятся не сразу. Оставьте шлюз поработать на полчаса, чтобы увидеть динамику.
3. **MQTT-топик:** Теперь шлюз публикует данные в топик `smart/weather-gate/state` (или с тем именем, которое вы задали в настройках). Это упрощает интеграцию с НА, так как формат JSON остается стандартным для `rtl_433`.

Маленький нюанс по отображению:

Если на графиках вместо цифр вы видите «0» или пустые места, убедитесь, что в файле `index.html` в JavaScript-части названия массивов из `/api/history` совпадают с теми, что мы прописали в скетче:

- `temp_history`
 - `press_history`
 - `wind_history`
-

Что дальше?

Теперь, когда железная и программная части шлюза отлажены, можно:

- Добавить **авто-обнаружение (Discovery)** для Home Assistant, чтобы датчики сами появились в интерфейсе (код для этого есть в вашем `smart_counter` на стр. 17).
- Настроить **ОТА-обновление**, чтобы прошивать шлюз по сети.

Появились ли первые данные о температуре и влажности в интерфейсе после пролета радиопакетов?

подключил погодную станцию и её собственный монитор для проверки. с погодной станции данные идут. на её мониторе отображаются, через CC1101 данных не вижу

Раз монитор станции (база) видит данные, а шлюз — нет, значит, радиомодуль **CC1101** работает, но не "слышит" нужную частоту или протокол. У метеостанций Fine Offset (Weatherwise) есть пара нюансов, которые могут блокировать прием.

Вот чек-лист для проверки, от простых вещей к сложным:

1. Точная подстройка частоты (Frequency Offset)

Дешевые модули CC1101 часто имеют отклонение кварца. Если база станции настроена идеально, а CC1101 "смещен" на 10-20 кГц, он ничего не поймает.

•**Что сделать:** В `setup()` попробуйте немного подвигать частоту вверх-вниз с шагом 0.01-0.02 МГц: `915.02`, `915.04`, `914.98`.

•**Или:** В мониторе порта посмотрите на сообщения `Average RSSI`. Если там значение типа `-100 dbm` и оно не меняется при передаче (когда на базе мигает значок антенны), значит, мы "мимо" частоты.

2. Тип модуляции (OOK vs FSK)

Старые WH1080 работают в **OOK** (амплитудная манипуляция), новые — в **FSK** (частотная). Библиотека `rtl_433_ESP` по умолчанию пытается угадать, но иногда ей нужно помочь.

•**Проверка:** В файле `rtl_433_ESP.h` убедитесь, что `#define OOK_MODULATION true`. Если станция современная, попробуйте сменить на `false` (но для Fine Offset чаще нужен `true`).

3. Список протоколов

По умолчанию в библиотеке могут быть включены не все декодеры для экономии памяти.

•**Действие:** Найдите в библиотеке файл `rtl_433_ESP_Config.h` и убедитесь, что протокол **Fine Offset WH1080** (обычно ID 32 или 1) не закомментирован.

4. Пин GDO0 (Прерывание)

Это самая частая причина "тишины".

•**Проверка:** Пин **GDO0** модуля CC1101 должен быть подключен к **GPIO 4** (как в нашем скетче).

•**Тест:** Если у вас есть светодиод на `LED_RF_RX` (GPIO 25), вспыхивает ли он хоть иногда? Если он вообще не моргает, значит, на GPIO 4 не приходят импульсы от радио. Проверьте пайку и качество провода "дюпон".

5. Антенна

На частоте 915 МГц антенна — это критически важно.

•Простой кусок провода должен быть длиной ровно **8.2 см**.

•Если антенна спиральная (пружинка), убедитесь, что она не касается металлических частей или экрана Ethernet-порта.

Предложение: Давайте добавим в `loop` вывод текущего уровня шума (RSSI). Если при передаче данных станцией цифра в мониторе порта будет резко прыгать (например, с -100 до -60), значит, **радио часть работает**, и проблема только в **декодере** (программе).

Вспыхивает ли у вас **светодиод приема** на 25-м пине, когда станция передает данные?

Тот факт, что данные отображаются на базе, но не доходят до модуля **CC1101**, при условии исправности «железа», чаще всего связан с программными фильтрами или аппаратным рассогласованием частоты.

Исходя из документации библиотеки `rtl_433_ESP` (в PDF) и вашего текущего прогресса, вот пошаговый план отладки:

1. Проверка «сырого» сигнала (RSSI)

Если библиотека «молчит», нужно понять, видит ли CC1101 радиоэфир вообще.

- В мониторе порта должны идти строки: `Average RSSI Signal -91 dbm`.

- **Эксперимент:** Посмотрите на базу метеостанции. Когда на её экране мигает значок антенны (момент передачи), значение RSSI в мониторе порта должно резко измениться (например, стать `-60` или `-55`).

- **Если RSSI не меняется:** CC1101 физически «не слышит» передатчик. Проверьте антенну (длина 8.2 см) и не перепутан ли пин **GDO0** (в коде и PDF это **GPIO 4** (р. 1)).

2. Подстройка частоты (Frequency Offset)

У дешевых модулей CC1101 частота может «плавать». Попробуйте в `setup()` немного сместить частоту от центральной:

cpp

```
rf.initReceiver(4, 915.02); // Смещение +20 кГц
```

```
// или
```

```
rf.initReceiver(4, 914.98); // Смещение -20 кГц
```

Используйте код с осторожностью.

3. Режим отладки (Verbose)

Библиотека поддерживает вывод подробной информации о каждом пойманном импульсе. Это поможет понять, видит ли она протокол.

Добавьте в `setup()`:

cpp

```
rf.rtlVerbose = 1; // Уровень Verbose (согласно PDF стр. 9)
```

Используйте код с осторожностью.

Если после этого при передаче данных в консоли поползут строки типа `Pulse train...`, значит, радиочасть в порядке, но **декодер** не узнает протокол WH1080.

4. Протокол и Модуляция

В присланном PDF (стр. 9 (р. 9)) указано, что переменная `ookModulation` по умолчанию `true`. Станции Fine Offset (Weatherwise) обычно работают именно в OOK.

- Убедитесь, что в вашем коде **НЕ** вызываются методы принудительного перехода в FSK.
- Проверьте файл `rtl_433_ESP_Config.h`. Там должен быть включен декодер именно для вашей модели.

5. Питание и Помехи

WT32-ETH01 создает сильные наводки через Ethernet-трансформатор.

•**Совет:** Попробуйте физически отодвинуть модуль CC1101 от разъема Ethernet на максимальное расстояние проводов.

•Убедитесь, что CC1101 запитан от стабильных **3.3В**. Если напряжение проседает ниже 3.0В, чувствительность приемника падает в десятки раз.

Предлагаю следующее: включите `rf.rtlVerbose = 1` и посмотрите, появятся ли в мониторе порта записи об обнаруженных «импульсах» (Pulse trains) в моменты, когда база станции обновляет данные. Если импульсы есть, нам нужно будет просто подправить настройки декодера.

```
ets Jul 29 2019 12:21:46 rst:0x3 (SW_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT) configsip: 0,
SPIWP:0xee      clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO,  clock div:1  load:0x3fff0030,len:4876  ho 0 tail 12 room 4
load:0x40078000,len:16560  load:0x40080400,len:3500  entry 0x400805b4 rtl_433_ESP(7):
Average RSSI Signal -91 dbm, adjusted RSSI Threshold -82, samples 50000 rtl_433_ESP(7):
Average RSSI Signal -91 dbm, adjusted RSSI Threshold -82, samples 50000 rtl_433_ESP(7):
Average RSSI Signal -91 dbm, adjusted RSSI Threshold -82, samples 50000 rtl_433_ESP(7):
Average RSSI Signal -91 dbm, adjusted RSSI Threshold -82, samples 50000
```

Постоянный уровень **-91...-92 dBm** в мониторе порта говорит о том, что радиомодуль CC1101 видит только фоновый шум эфира. В моменты передачи данных от

метеостанции (которые вы видите на родном мониторе) эти цифры **обязаны** прыгать до значений **-60...-50 dBm**.

Если цифры стоят на месте, у нас есть две вероятные причины:

1. Проблема с антенной или частотой

Модули CC1101 на 915 МГц крайне чувствительны.

- Частота:** Попробуйте в `setup` изменить частоту на **868.3 МГц** или **433.92 МГц**. В России метеостанции WH1080 чаще всего работают на **868 МГц**, даже если на коробке написано иное.

- Антенна:** Проверьте пайку антенны к модулю. На 915 МГц обычный провод должен быть длиной строго **82 мм**, на 868 МГц — **86 мм**.

2. Пин GDO0 (GPIO 4)

Если на GPIO 4 (GDO0) «тишина», библиотека считает, что сигнала нет.

- Убедитесь, что провод от CC1101 (пин GDO0) идет именно на **GPIO 4**.

- Посмотрите на светодиод на плате CC1101 (если он есть): мигает ли он в момент передачи?

3. Тест «на лету» (Поиск частоты)

Чтобы понять, «слышит» ли шлюз станцию, добавьте в `setup` команду для вывода всех импульсов (режим отладки):

cpp

```
rf.rtlVerbose = 1;
```

Используйте код с осторожностью.

Если после этого в мониторе по-прежнему только строки Average RSSI, а импульсы (Pulse train...) не появляются — значит, CC1101 физически не видит сигнал.

Попробуйте изменить частоту на 868.3 МГц:

cpp

```
rf.initReceiver(4, 868.3);
```

Используйте код с осторожностью.

Меняются ли цифры RSSI в мониторе порта в момент, когда станция передает данные на свой родной монитор?

на мониторе погодной станции, на заднем шильдике, написано: Weather Station (receiver) Model: WS1080 Power:3x1,5 AA battery Frequency^915 MHz FCC ID WA5915MTX

Если на шильдике четко указано **915 MHz**, значит, мы ищем в правильном диапазоне, но «промахиваемся» мимо точного пика частоты или модуляции. Значение **-91 dBm** в логах подтверждает, что CC1101 работает, но сигнал от станции для него не выделяется

из общего шума.

Почему шлюз не видит станцию на 915 МГц:

1.Смещение частоты (Frequency Offset):

Кварцевые резонаторы на дешевых модулях CC1101 часто имеют отклонение. Станция передает на 915.000, а ваш модуль из-за погрешности «слушает» на 915.020. Для цифрового сигнала это критично.

•**Решение:** Попробуйте в `setup()` перебрать частоту с мелким шагом.

Измените `rf.initReceiver(4, 915.0);` последовательно на:

•914.95

•914.98

•915.02

•915.05

2.Порог чувствительности (RSSI Threshold):

В логе написано: `adjusted RSSI Threshold -82`. Это значит, что библиотека игнорирует всё, что слабее -82 dBm. Если станция стоит далеко или антенна слабая, сигнал может приходить на уровне -85 dBm и просто отсекается.

•**Решение:** Положите датчик метеостанции вплотную к шлюзу (на расстоянии 1 метра).

Если RSSI не прыгнет до -50...-40, значит, проблема в антенне или согласовании.

3.Антенна и помехи от Ethernet:

WT32-ETH01 — очень «шумная» плата. Ethernet-контроллер наводит сильные помехи именно в диапазоне 900 МГц.

•**Решение:** Если CC1101 припаян вплотную к плате, попробуйте вынести его на проводах длиной 10-15 см подальше от разъема Ethernet.

Что сделать сейчас для проверки:

Добавьте в `setup()` одну строку для принудительного вывода импульсов (как указано в вашем PDF на стр. 9):

cpp

```
rf.rtlVerbose = 1;
```

Используйте код с осторожностью.

Если после этого в мониторе порта:

•**По-прежнему только RSSI:** CC1101 физически не ловит несущую (проблема в антенне, питании или смещении частоты).

•**Появились строки `Pulse train...`:** Радио работает, импульсы идут, но **декодер** не может собрать из них пакет. Тогда нужно будет подправить тайминги в библиотеке.

Попробуйте поднести датчик вплотную к шлюзу. Прыгает ли значение RSSI в мониторе порта в момент передачи? (Обычно датчик WH1080 передает данные раз в 48 секунд, на нем в этот момент мигает светодиод).

Совет: проверьте питание CC1101. Если на нем меньше 3.0В, он может «видеть» шум, но не иметь чувствительности для приема данных.

Погодная станция лежит на столе в полуметре от CC1101, рядом с монитором станции (для контроля). антенна прикручена маленькая, 4см, но на таком расстоянии это не имеет значение. по поводу питания, стоит DC-DC преобразователь 3.3 вольта 3 ампера. напряжение по факту 3.28 вольт. если надо могу чуть увеличить.

С питанием у вас полный порядок — 3.28В при такой мощности DC-DC более чем достаточно, увеличивать не нужно. Антенна 4 см на таком расстоянии тоже должна работать.

Раз RSSI стоит на месте при датчике в полуметре, значит, **модуль CC1101 «ослеп»** или работает не в том режиме. Причина почти наверняка в том, что библиотека по умолчанию настроена на частоту 433 МГц и при переключении на 915 МГц «криво» выставляет параметры чувствительности или модуляции.

Давайте сделаем «шоковую терапию» для CC1101, чтобы понять, живой он или нет.

1. Тест на «всеядность» (Снижение порога)

Добавьте в `setup()` после `rf.initReceiver` эти строки. Они заставят библиотеку видеть даже самые слабые сигналы:

```
rf.setRSSIThreshold(-95); // Максимальная чувствительность
rf.rtlVerbose = 1;        // Видеть каждый «чих» в эфире
```

2. Подстройка частоты (Самое вероятное)

FCC ID `WA5915MTX` подтверждает, что станция работает на 915 МГц. Но CC1101 требует точной настройки. Попробуйте в `setup()` сделать такой цикл (только для теста!), чтобы найти частоту:

```
// Вместо одной частоты, попробуйте по очереди (перешивая):
rf.initReceiver(4, 915.00);
// Если нет данных через 2 минуты, попробуйте:
// rf.initReceiver(4, 914.90);
// rf.initReceiver(4, 915.10);
```

3. Проверка GPIO 4 (GDO0)

Это критический момент. Если в мониторе порта вообще нет строк `Pulse train`, значит, пин **GDO0** на CC1101 не выдает импульсы на ESP32.

- На модуле CC1101 есть пин **GDO0**. Проверьте мультиметром (или осциллографом), прыгает ли на нем напряжение в момент вспышки светодиода на датчике метеостанции.
- Если на GDO0 всегда 0 или всегда 3.3В — модуль не сконфигурирован.

4. Прямая правка модуляции

В вашем PDF (стр. 10) упоминается переменная `ookModulation`. Для WH1080 она должна быть `true`. Иногда библиотека «забывает» это при смене частоты.

Добавьте в `setup()`:

```
rf.ookModulation = true;
```

Что сейчас нужно увидеть в мониторе порта:

Если после добавления `rf.rtlVerbose = 1` при вспышке на датчике в мониторе порта **НЕ** посыпались строки с цифрами (таймингами импульсов), значит:

1. Либо CC1101 не настроился на прием (ошибка в SPI или командах).
2. Либо GPIO 4 не видит сигнал (проблема в проводе GDO0).

Попробуйте добавить `rf.rtlVerbose = 1` и `rf.setRSSIThreshold(-95)`. Появится ли хоть какая-то новая информация в мониторе порта в момент вспышки на датчике?

P.S. Если есть возможность, попробуйте временно отключить Ethernet кабель (питая плату от БП) — просто чтобы исключить дикие наводки от LAN-трансформатора на таком близком расстоянии.

Напишите, изменился ли лог после включения Verbose.

а есть ли возможность добавить в скетч перестройку частоты "на горячую", чтобы было поле, куда я буду вводить частоты и таким образом не перешивая скетч "гулять" вверх/вниз немного по частоте?

Да, это отличная идея для отладки. Так как у нас уже есть веб-сервер и API, мы можем добавить поле ввода частоты прямо на веб-страницу. Это позволит вам «сканировать» диапазон в реальном времени, не перезаписывая скетч.

Для этого нам нужно:

1. Добавить в API `/api/config` обработку параметра частоты.
2. Реализовать в коде функцию перенастройки `rf.initReceiver` «на лету».
3. Добавить в HTML-код поле ввода и кнопку.

1. Изменения в скетче (C++)

Вам нужно найти в скетче обработку API для сохранения конфига (или добавить новую) и добавить переменную `current_freq`.

```
// Добавьте глобальную переменную
float current_freq = 915.0;

// В секции setup(), где описаны эндпоинты сервера, добавьте/обновите:
server.on("/api/set_freq", HTTP_POST, []() {
    if (server.hasArg("f")) {
        current_freq = server.arg("f").toFloat();

        // Переинициализируем приемник на новой частоте
        rf.disableReceiver();
        rf.initReceiver(4, current_freq);
        rf.enableReceiver();

        Serial.printf("[RF] Frequency changed to: %.2f MHz\n", current_freq);
        server.send(200, "text/plain", "OK");
    } else {
        server.send(400, "text/plain", "Missing frequency");
    }
});
```

2. Изменения в `index.html` (JS)

Добавьте в блок настроек в `index.html` поле для ввода частоты:

```
<!-- Внутри <div id="settingsContent"> -->
<div class="settings-row">
    <div>
        <label>Частота (MHz)</label>
        <input type="number" step="0.01" id="rf_freq" value="915.00">
    </div>
</div>
<button onclick="setFreq()">Применить частоту</button>
```

```
<script>
async function setFreq() {
  const f = document.getElementById('rf_freq').value;
  const res = await fetch('/api/set_freq?f=' + f, { method: 'POST' });
  if (res.ok) alert('Частота изменена на ' + f + ' MHz');
}
</script>
```

Как проводить поиск (Методика):

1. **Включите лог:** Убедитесь, что в коде стоит `rf.rtlVerbose = 1;`.
2. **Шаг поиска:** Начните с `915.00`. Подождите минуту (цикл датчика).
3. **Идите «змейкой»:** Если тишина, пробуйте `915.05`, затем `914.95`, `915.10`, `914.90`.
4. **Смотрите на RSSI:** Если при вводе определенной частоты фоновый RSSI в мониторе порта станет не `-91`, а, скажем, `-85`, значит, вы поймали какой-то шум на этой частоте. Нам нужно найти ту точку, где при вспышке светодиода на датчике будет максимальный «всплеск» (например, до `-50`).

Почему это поможет:

Кварцевые резонаторы на модулях CC1101 за 2-3 доллара часто имеют погрешность до 30-50 кГц. С помощью этого «ползунка» вы найдете реальный резонанс вашей пары «датчик-приемник».

Сможете добавить эти строки в свои API и HTML, чтобы начать поиск «в прямом эфире»?

нужен полный текст скетча. и, скорее всего, ещё и html

Для того чтобы вы могли «гулять» по частоте в реальном времени, я модифицировал скетч: добавил переменную частоты в `Preferences` (чтобы она сохранялась после перезагрузки) и создал API-эндпоинт для её смены «на горячую».

1. Полный текст скетча (v4.7.0-Scanner)

Этот код включает в себя всё: Ethernet, MQTT, RTC, Web-сервер, LittleFS и функцию смены частоты.

```
#define MQTT_MAX_PACKET_SIZE 1024
#include <ETH.h>
#include <WiFi.h>
#include <SPI.h>
```

```

#include <Wire.h>
#include <RTCLib.h>
#include <LittleFS.h>
#include <ArduinoJson.h>
#include <WebServer.h>
#include <PubSubClient.h>
#include <Preferences.h>
#include <rtl_433_ESP.h>

// ===== КОНФИГУРАЦИЯ =====
#define FW_VERSION "4.7.0-Scanner"
#define LED_STATUS 5
#define ETH_POWER_PIN 16
#define I2C_SDA 32
#define I2C_SCL 33

// Глобальные объекты
RTC_DS3231 rtc;
WiFiClient ethClient;
PubSubClient mqtt(ethClient);
rtl_433_ESP rf;
WebServer server(80);
Preferences pref;

// Переменные данных
float temperature = 0, humidity = 0, wind_speed = 0, current_freq = 915.0;
unsigned long last_packet = 0;
String hostname = "weather-gate";
String mqtt_ip = "";
char msgBuffer[512];

// ===== ФУНКЦИИ =====

void rtl_433_Callback(char* message) {
    digitalWrite(LED_STATUS, HIGH);
    JsonDocument doc;
    if (!deserializeJson(doc, message)) {
        temperature = doc["temp_C"] | temperature;
        humidity = doc["humidity"] | humidity;
        wind_speed = doc["wind_avg_m_s"] | wind_speed;
        last_packet = millis();
    }
    if (mqtt.connected()) mqtt.publish(("smart/" + hostname + "/state").c_str(),
message);
}

```

```

    delay(30);
    digitalWrite(LED_STATUS, LOW);
}

bool handleFileRead(String path) {
    if (path.endsWith("/")) path += "index.html";
    String contentType = "text/html"; // Упростим для теста
    if (path.endsWith(".js")) contentType = "application/javascript";
    if (path.endsWith(".css")) contentType = "text/css";

    if (LittleFS.exists(path)) {
        File file = LittleFS.open(path, "r");
        server.streamFile(file, contentType);
        file.close();
        return true;
    }
    return false;
}

void setup() {
    Serial.begin(115200);
    pinMode(LED_STATUS, OUTPUT);
    pinMode(ETH_POWER_PIN, OUTPUT);
    digitalWrite(ETH_POWER_PIN, HIGH);

    SPI.begin(12, 15, 14, 2);
    Wire.begin(I2C_SDA, I2C_SCL);
    rtc.begin();
    LittleFS.begin(true);

    // Загрузка частоты и настроек
    pref.begin("weather", false);
    current_freq = pref.getFloat("freq", 915.0);
    hostname = pref.getString("hostname", "weather-gate");
    mqtt_ip = pref.getString("mq_ip", "");
    pref.end();

    WiFi.mode(WIFI_OFF);
    ETH.begin(ETH_PHY_LAN8720, 1, 23, 18, 16, ETH_CLOCK_GPIO17_OUT);
    ETH.setHostname(hostname.c_str());

    // --- API ---
    server.on("/api/data", HTTP_GET, []() {
        JsonDocument doc;

```



```

        doc["temperature"] = temperature;
        doc["humidity"] = humidity;
        doc["freq"] = current_freq;
        doc["uptime"] = millis() / 1000;
        doc["last_update"] = last_packet > 0 ? (millis() - last_packet)/1000 : -1;
        String buf; serializeJson(doc, buf);
        server.send(200, "application/json", buf);
    });

    server.on("/api/set_freq", HTTP_GET, []() {
        if (server.hasArg("f")) {
            current_freq = server.arg("f").toFloat();
            pref.begin("weather", false);
            pref.putFloat("freq", current_freq);
            pref.end();

            rf.disableReceiver();
            rf.initReceiver(4, current_freq);
            rf.enableReceiver();

            Serial.printf("[RF] Frequency changed to: %.2f MHz\n", current_freq);
            server.send(200, "text/plain", "OK");
        }
    });

    server.onNotFound([]() {
        if (!handleFileRead(server.uri())) server.send(404, "text/plain", "404");
    });

    server.begin();

    // Инициализация радио
    rf.setCallback(rtl_433_Callback, msgBuffer, 512);
    rf.rtlVerbose = 1; // ВКЛЮЧАЕМ ЛОГ ИМПУЛЬСОВ
    rf.initReceiver(4, current_freq);
    rf.enableReceiver();

    if (mqtt_ip.length() > 5) mqtt.setServer(mqtt_ip.c_str(), 1883);
}

void loop() {
    server.handleClient();
    rf.loop();
    if (!mqtt.connected() && mqtt_ip.length() > 5) {

```

```

    static uint32_t lastM = 0;
    if (millis() - lastM > 10000) { lastM = millis();
mqtt.connect(hostname.c_str()); }
    }
    mqtt.loop();
}

```

2. Полный текст `index.html` (с полем для частоты)

Этот файл нужно положить в папку `data` и загрузить в `LittleFS`.

```

<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Scanner Gateway</title>
  <style>
    body { font-family: sans-serif; background: #f0f2f5; padding: 20px; text-align:
center; }
    .card { background: white; padding: 20px; border-radius: 15px; box-shadow: 0
4px 10px rgba(0,0,0,0.1); display: inline-block; min-width: 300px; }
    h1 { color: #1a73e8; }
    .val { font-size: 32px; font-weight: bold; margin: 10px 0; }
    .freq-input { margin-top: 20px; padding: 10px; border-top: 1px solid #eee; }
    input { width: 80px; padding: 8px; border-radius: 5px; border: 1px solid #ccc;
font-size: 16px; }
    button { padding: 8px 15px; background: #1a73e8; color: white; border: none;
border-radius: 5px; cursor: pointer; margin-left: 5px; }
    .info { font-size: 12px; color: #666; margin-top: 10px; }
  </style>
</head>
<body>
  <div class="card">
    <h1>Weather Gate</h1>
    <div class="val"><span id="temp">0.0</span> °C</div>
    <div>Влажность: <span id="hum">0</span> %</div>
    <div class="info">Обновлено: <span id="upd">--</span> сек. назад</div>
    <div class="freq-input">
      <label>Частота: </label>

```

```

        <input type="number" id="freq" step="0.01" value="915.00">
        <button onclick="applyFreq()">OK</button>
        <div class="info">Текущая: <b id="cur_freq">915.00</b> MHz</div>
    </div>
</div>

<script>
    async function updateData() {
        try {
            const res = await fetch('/api/data');
            const d = await res.json();
            document.getElementById('temp').innerText = d.temperature.toFixed(1);
            document.getElementById('hum').innerText = d.humidity;
            document.getElementById('cur_freq').innerText = d.freq.toFixed(2);
            document.getElementById('upd').innerText = d.last_update >= 0 ?
d.last_update : "--";
        } catch (e) {}
    }

    async function applyFreq() {
        const f = document.getElementById('freq').value;
        await fetch('/api/set_freq?f=' + f);
        updateData();
    }

    setInterval(updateData, 5000);
    updateData();
</script>
</body>
</html>

```

Инструкция по поиску:

1. Загрузите скетч, загрузите LittleFS.
2. Откройте веб-страницу по IP-адресу шлюза.
3. Откройте **Serial Monitor** в Arduino IDE.
4. Вводите частоту в поле (например, 915.05) и жмите OK.
5. Смотрите в Serial Monitor. При каждой вспышке светодиода на датчике метеостанции (раз в 48 сек) вы должны увидеть либо **всплеск RSSI** (с -91 до -50), либо появление строк **Pulse train...**
6. Если вы увидели **Pulse train**, значит частота верная — оставайтесь на ней, шлюз скоро декодирует данные.

Попробуем найти «тот самый» пик частоты? Проверьте, что в мониторе порта при `rf.rtlVerbose = 1` начали бежать данные при смене частоты.

что-то пошло не так. скетч прошил, страничку загрузил. в мониторе порта вот это: ets
Jul 29 2019 12:21:46 rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT) configsip:
0, SPIWP:0xee clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1 load:0x3fff0030,len:4876 ho 0 tail 12 room 4
load:0x40078000,len:16560 load:0x40080400,len:3500 entry 0x400805b4 rtl_433_ESP(3):
CC1101 radio.begin() failed, code: -2

Ошибка `code: -2` (Chip not found) означает, что ESP32 не смогла «увидеть» модуль CC1101 по шине SPI. Поскольку до этого шлюз работал, проблема возникла из-за конфликта инициализации **SPI** и **I2C (RTC)**.

В WT32-ETH01 шина SPI (пины 12, 14, 15) очень чувствительна к порядку включения. В последнем скетче не была явно вызвана команда `SPI.begin()`, из-за чего библиотека RadioLib (внутри `rtl_433_ESP`) не смогла достучаться до чипа.

Как исправить:

Замените блок `setup()` в вашем скетче на этот вариант. Здесь мы принудительно запускаем SPI **до** всего остального.

cpp

```
void setup() {
```

```
Serial.begin(115200);
```

```
delay(500);
```

```
Serial.println("\n--- STARTING SYSTEM ---");
```

```
pinMode(LED_STATUS, OUTPUT);
```

```
pinMode(ETH_POWER_PIN, OUTPUT);
```

```
digitalWrite(ETH_POWER_PIN, HIGH);
```

```
// 1. ПРИНУДИТЕЛЬНЫЙ ЗАПУСК SPI (для CC1101)
```

```
// Важно: SCK=12, MISO=15, MOSI=14, SS=2
```

```
SPI.begin(12, 15, 14, 2);
```

```
// 2. ЗАПУСК I2C (для RTC)
```

```
Wire.begin(I2C_SDA, I2C_SCL);
```

```
rtc.begin();
```

```
// 3. ФАЙЛОВАЯ СИСТЕМА И НАСТРОЙКИ
```

```
if (!LittleFS.begin(true)) Serial.println("FS Error");
```

```
pref.begin("weather", false);
```

```
current_freq = pref.getFloat("freq", 915.0);
```

```
hostname = pref.getString("hostname", "weather-gate");
```

```
mqtt_ip = pref.getString("mq_ip", "");
```

```
pref.end();
```

```
// 4. CEB
```

```
WiFi.mode(WIFI_OFF);
```

```
btStop();
```

```
ETH.begin(ETH_PHY_LAN8720, 1, 23, 18, 16, ETH_CLOCK_GPIO17_OUT);
```

```
ETH.setHostname(hostname.c_str());
```

```
// 5. API
```

```
server.on("/api/data", HTTP_GET, []() {
```

```
    JsonDocument doc;
```

```
    doc["temperature"] = temperature;
```

```
    doc["humidity"] = humidity;
```

```
    doc["freq"] = current_freq;
```

```
    doc["uptime"] = millis() / 1000;
```

```
    doc["last_update"] = last_packet > 0 ? (millis() - last_packet)/1000 : -1;
```

```
    String buf; serializeJson(doc, buf);
```

```
    server.send(200, "application/json", buf);
```

```
});
```

```
server.on("/api/set_freq", HTTP_GET, []) {
```

```
    if (server.hasArg("f")) {
```

```
        current_freq = server.arg("f").toFloat();
```

```
        pref.begin("weather", false);
```

```
        pref.putFloat("freq", current_freq);
```

```
        pref.end();
```

```
        rf.disableReceiver();
```

```
        rf.initReceiver(4, current_freq);
```

```
        rf.enableReceiver();
```

```
        server.send(200, "text/plain", "OK");
```

```
    }
```

```
});
```

```
server.onNotFound([]) {
```

```
    if (!handleFileRead(server.uri())) server.send(404, "text/plain", "404");
```

```
});
```

```
server.begin();
```

```
// 6. РАДИО (Теперь SPI готов)
```

```
rf.setCallback(rtl_433_Callback, msgBuffer, 512);
```

```
rf.rtlVerbose = 1;
```

```
rf.initReceiver(4, current_freq);
```

```
rf.enableReceiver();
```

```
if (mqtt_ip.length() > 5) mqtt.setServer(mqtt_ip.c_str(), 1883);
```

```
Serial.println("--- SETUP COMPLETE ---");
```

```
}
```

Используйте код с осторожностью.

На что обратить внимание:

1. **Пин 2 (CS):** Если вы физически переподключали модуль, убедитесь, что пин **CS (Chip Select)** модуля CC1101 идет именно на **GPIO 2**.
2. **Пин 4 (GDO0):** Пин прерываний должен идти на **GPIO 4**.
3. **Перемычки:** Если вы использовали GPIO 0 для прошивки, убедитесь, что **сняли перемычку** перед тестом, иначе SPI может работать некорректно.

Удалось ли теперь пройти инициализацию радио без ошибки code: -2? Если в мониторе порта появилось --- SETUP COMPLETE ---, значит, можно начинать поиск частоты через веб-страницу.